

7th Progress Report

W. Chan

Date:	07/03/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Changed my mind that a pure IDHP controller is not as ‘fault tolerant’ as an online-offline hybrid or a drl controller. IDHP as an adaptive controller offers its own special fault tolerancy since it can change controller parameters as a result of any system dynamics changes...
- Mentioned I am more inclined to research improving IDHP, so will push to continue this path if eligibility traces seem to show noticeable/significant improvement.

Description of the results since last meeting

- Playing with the Flying-V dynamics from Gitlab, found the pre-existing linearizing scripts and got them working. It gives a linear model modelling the dynamics of:

$$p_{ref}, \delta n_{z,ref}, \beta_{ref} \rightarrow \phi, \beta, q, n_z, \theta \quad (1)$$

This linear model isn’t useful for me since it linearizes a Flying-V with Jurian’s controller + sensor dynamics, and also the input-output relation is irrelevant to my work. I think I’ll use this script instead to see how I can linearise the dynamics of *only* the Flying-V. But it’s interesting to see that he kind of uses frequency-domain system ID to do model order reduction.

- **Decision on preliminary model and test scenarios:**

Regarding what linear model to use, I narrowed it down to two options:

1. Linearize for short-period, find linear model from: elevator $\rightarrow \alpha$ and q .
 - Pros: More logical to do reference following with such model as we can follow a pitch rate reference, very simple as only one input and two outputs, several past theses and research used this model (Casper, Stefan, Zhou, Peter) so can do some eye norm evaluation.
 - Cons: Pitching motion is less troublesome on the flying-v than yawing motions so this model is theoretically less important than a yawing model.
2. Linearize for dutch-roll/yawing motion, find linear model from: rudder $\rightarrow \beta$ and r .
 - Pros: Directly models simplified Dutch roll which is unstable in the flying-v, also only has 1 input and two outputs.

- Cons: logically can only do regularization tasks since *i think* you don't typically try to command a sideslip angle unless maybe during landing, so maybe little bit less freedom in what logical control problems can be designed. Not been used in any previous research (as far as I could tell), so less consistency/comparison potential.

To make a decision between which of these linear models to use, I looked at the fact that this model is simply to provide a basis for preliminary analysis for the algorithm. This detracts a little from the Pros of using a DR model since I will emphasize evaluating dutch roll stability/damping in the second research phase anyways. So I've chosen to **use the short-period** linear model instead. Meaning I will linearise the dynamics to find the linear model between the following input and outputs:

$$u = [\delta_e]$$

$$y = [\alpha \quad q]^\top$$

To complete the control problem, the nominal task will be simply to track a sinusoidal pitch rate reference signal, just like in Zhou's IDHP paper, and in Casper's work, amongst others.

The faulty scenario will have a mix of:

- smaller values for the input matrix to simulate a damaged elevator,
 - flip sign in the input matrix to simulate FCS fault,
 - and then some completely hypothetical faults such as randomly changing signs in the state space matrix elements.
- Moved on to trying to trim Flying-V dynamics, and managed to obtain a linear model linearized around a trim point Jurian derived. But strangely, Jurian's trim point did not put the flying-v at equilibrium, even though his scripts looked bug-free. After some debugging and head-scratching, I finally asked for help from Direnc, and then with more head-scratching together, we came up with a solution for trimming the flying V which got the dynamics to go from completely unstable to marginally unstable. The timeline of trimming went as follows:
 1. Linearized Flying-V dynamics *on the first day*, realised trim point was unstable when model matching nonlinear and linear model.
 2. Tried to debug trim point computation scripts, and realised they were bug-free and should work.
 3. Kept trying to change flight conditions and optimization algorithms, even got an EA optimizer to try trimming because I thought maybe fminsearch was not global enough to find trim point, nothing worked.
 4. After a long time finally admitted defeat and asked Direnc for help, realised I was trimming for the unstable approach condition.
 5. Switched to trimming for cruise, still the trim routines can't find stable points.
 6. Swapped the Simulink model for entire flying v dynamics, debugged the models a little, trim routine can finally find trim point that is marginally unstable.
 7. Did some refactoring of the model and suddenly the dynamics became perfectly trimmed.

Finally, managed to arrive at the short-period linear model with the actuator dynamics modelled (did not do this intentionally, forgot that I included these dynamics in Simulink when linearizing). A model match checking plot between the linear and nonlinear system is shown in ??, which shows that the linear and nonlinear model shows good agreement in the input signals tested, note as the impulse input was scaled up, the responses began diverging from each other, but when the input is extended over time, the responses converge.

I then coded this SP model into a gymnasium-like class in Python.

- Starting work on implementing IDHP, using mainly Casper's work as a reference. Right now I feel like his code is very verbose for seemingly no reason, but maybe I'll discover soon enough why this is. I've also never used tensorflow before so I'm excited to learn how to use it.

So far I have implemented the critic and actor networks, at the moment they are very simple implementations, both are simply a tensorflow.model object wrapped in a small custom class object I made. Still need to implement the RLS estimator and combine the 3 components into an IDHP script/class.

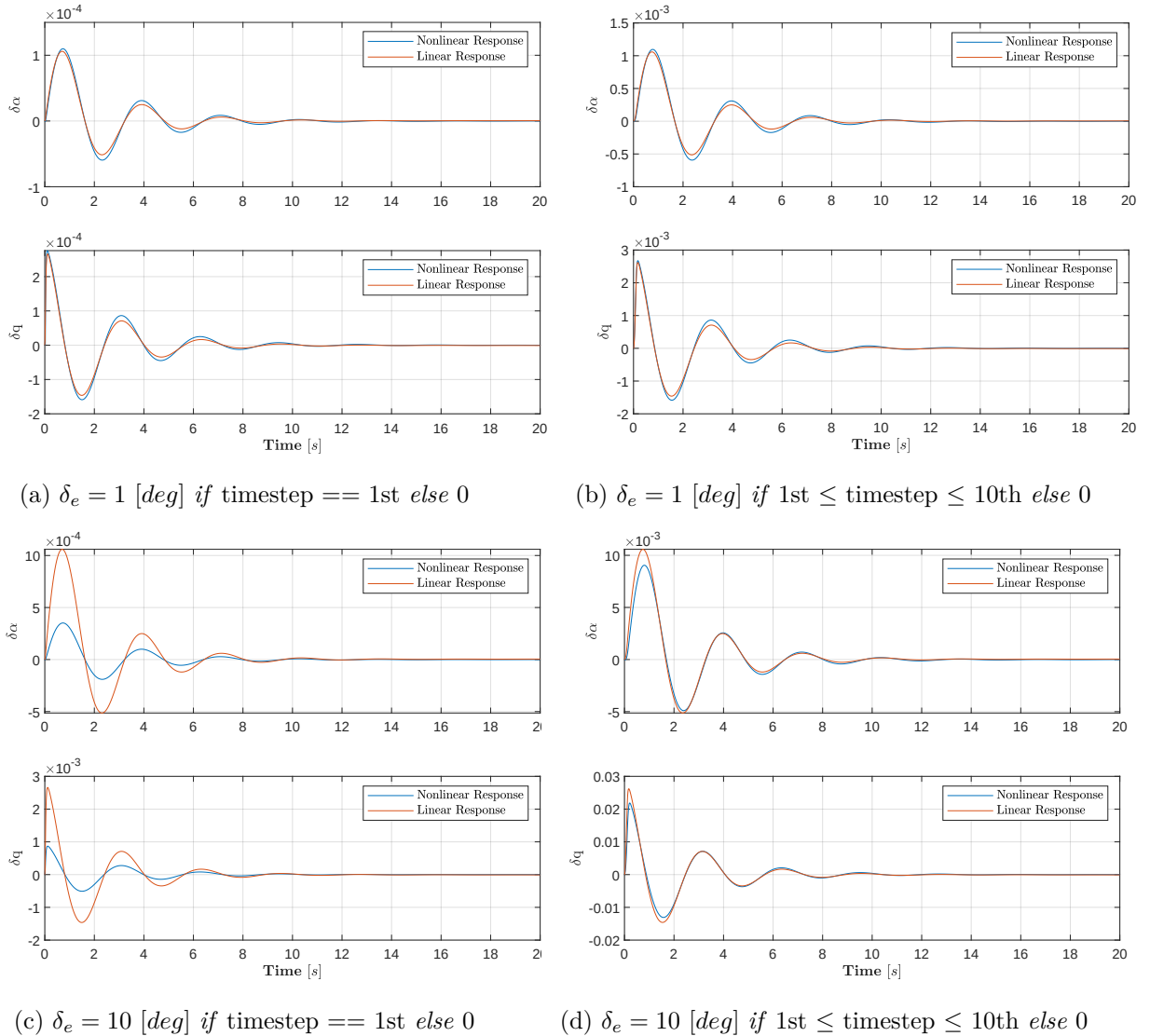


Figure 1: Plot of the $\alpha [deg]$ and $q [deg/s]$ deviations from both the linear and nonlinear system 'impulse' and step responses, $dt = 0.01 [s]$.

Plans for next 2 weeks

1. Work on implementing the RLS estimator, once this is done I can work on implementing the full IDHP algorithm script, e.g. working on the update rules for the critic and actor.
2. Refactor some of the Matlab and Python scripts I have written thus far, and make sure I write proper docstrings!!!